AI MOVIE RECOMMENDATION SYSTEM

Project Report

1.Introduction:

The AI Movie Recommendation System is a project that suggests movies to a user based on the movie they choose. The main idea is to find movies that are similar in terms of their genres, keywords, and description. I developed this project using Python and machine learning libraries. The system uses a movie dataset containing information such as genres and keywords. Python, Pandas, and Scikit-learn are used to process the data and build the recommendation system. CountVectorizer is used to convert movie features into numerical vectors, and cosine similarity is used to identify similar movies. Based on the selected movie, the system provides a list of movies that the user may like.

2. Methodology:

First, I loaded a movie dataset using Pandas. The dataset contains information about different movies. The methodology of the AI Movie Recommendation System follows a content-based approach. Movie features such as genres and keywords are processed and combined to identify similarities between movies. The processed information is converted into numerical vectors using CountVectorizer, and cosine similarity is used to find movies that are most similar. Based on these similarity scores, the system recommends the top matching movies to the user.

2.1 Data Selection

I selected the important columns needed for the recommendation system, including movie title, genres, keywords, and overview. The relevant movie data is selected from the dataset based on features such as genres and keywords. These features are useful for identifying similarities between movies and generating meaningful recommendations. Unnecessary columns are excluded so that the system can focus on the information required for the recommendation process.

2.2 Creating Movie Tags

I combined the genres, keywords, and movie overview into one text field called tags. This gave the system useful information about each movie in one place. Movie features are combined into tags to capture the main characteristics of each movie.

2.3 Text Vectorization

I used CountVectorizer from Scikit-learn to convert the movie tags into numerical vectors. This allows the computer to compare the movie information. Uses CountVectorizer from Scikit-learn. Helps the system represent movie tags in a form that the machine can process. These vectors are then used to calculate similarity between movies.

2.4 Cosine Similarity

I used cosine similarity to calculate how similar the movies are to each other. Movies with more similar information get a higher similarity score. Measures how similar two movies are by comparing their numerical feature vectors. A higher similarity score indicates that the movies have more similar content.

2.5 Recommendation Function

Finally, I created a recommendation function. When a movie title is entered, the system finds that movie and displays five movies that are most similar to it.

2.6 Top Recommendations

Display the most similar movies as recommendations. The system generates a list of top movie recommendations based on the similarity scores calculated between movies. It identifies the movies that are most similar to the movie selected by the user and displays them as recommendations. This helps users discover new movies that match the content and characteristics of their selected movie.

2.7 Testing

Test the system with different movie titles to verify the recommendations. The system is tested with different movie titles to evaluate the accuracy and relevance of the recommended movies.

3. Technologies Used:

Python, Pandas, Scikit-learn, CountVectorizer, and Cosine Similarity were used to build this project. The project was developed and tested in Google Colab using a Jupyter Notebook.

- Python – Main programming language used to develop the system.
- Pandas – Used for loading, processing, and managing the movie dataset.
- NumPy – Used for numerical and array-based operations.
- Scikit-learn – Used for machine learning and similarity calculations.
- CountVectorizer – Converts movie tags into numerical vectors.
- Cosine Similarity – Measures similarity between movies.
- Jupyter Notebook – Used to develop, run, and test the project.

4. Results:

The system successfully generated movie recommendations for different input movies. I tested the system with movies such as Avatar, Inception, The Dark Knight, The Matrix, Jurassic Park, and Titanic. For each movie, the system displayed five movies that were considered similar. It displays a list of top similar movies based on cosine similarity. The recommendations are generated from the movie features and tags in the dataset. Testing with different movies shows that the system can provide relevant movie recommendations.

5. Insights:

While working on this project, I learned how movie information can be used to build a simple recommendation system. I also learned how text data can be converted into numerical values and how cosine similarity can be used to compare items. I noticed that some movies can appear in more than one recommendation list because they have similar features. The project shows that movie features such as genres and keywords can be effectively used to identify similarities between movies. The recommendation system provides relevant suggestions based on the selected movie, demonstrating how machine learning and text-based techniques can be used to create a simple and useful movie recommendation system.

6. Conclusion:

The AI Movie Recommendation System successfully recommends movies based on their similarity to a movie selected by the user. This project helped me understand data preprocessing, text vectorization, similarity calculation, and the basic idea behind recommendation systems. It also gave me practical experience using Python and Scikit-learn. The AI Movie Recommendation System provides an efficient way to discover movies by analyzing the characteristics of existing movies. By combining relevant features and applying text vectorization and cosine similarity, the system identifies movies with similar content and presents them as recommendations. The successful testing of the system demonstrates that simple machine learning techniques can be effectively applied to real-world recommendation problems. This project also provides a foundation for developing more advanced recommendation systems in the future.

7. Future Improvements:

In the future, the system could be improved by using user ratings, watch history, or more movie features. A graphical user interface could also be added to make the system easier to use. The system can be improved by using more movie features such as cast, directors, ratings, and user preferences to provide more accurate recommendations. A larger and more updated dataset can also be used to improve the quality of results. The system could be enhanced with a user-based recommendation feature, allowing recommendations to be personalized according to individual interests. A user-friendly web or mobile interface could also be developed to make the recommendation system easier to use.